Programming for Robotics: Project Report

ATLAS: Autonomous Tracking Laser Aiming System

Author 1:	Madeline Abio s5259572
Author 2:	Alpa Sahai s5405889
GitHub Link:	https://github.com/alpasahai/programming_robotics_project.git
Specialisation Option Chosen:	Embedded Intelligence

Griffith University

Contents

List of Figures

1. Project Overview

The Autonomous Tracking and Laser Aiming System (ATLAS) is a simulated fire-control system that detects, tracks, and neutralises airborne threats without human input, operating in an outdoor Webots environment. It is split into two collaborative units: a Search Radar that sweeps a wide area with a wide-beam radar sensor, and a turret-mounted Fire-Control Radar (FCR) carrying a narrow-beam radar and projectile launcher. The two units coordinate over emitter/receiver pairs, with rotational motors driving the search sweep and the FCR's pan/tilt turret.

When the Search Radar spots a target, it cues the FCR, which locks on and fuses readings from both radars through a Kalman filter to smooth noisy measurements and predict the target's trajectory before firing. Alongside this, an online SGDClassifier learns the attacker's launch-direction pattern live via `partial_fit`, pre-slewing the turret toward the predicted next sector while idle and re-adapting continuously as the pattern drifts.

2. Technical Requirements Compliance

This section maps ATLAS to each technical requirement; details live in the cited sections. The figure and `FSM` label are defined here once and referenced elsewhere.

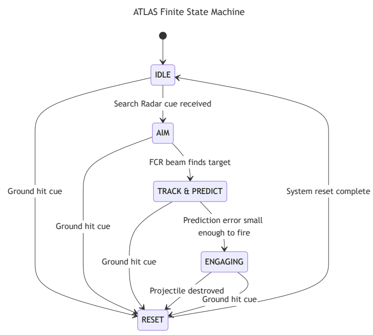

Figure 1: ATLAS finite state machine (FSM) diagram

- **Inputs (≥ 2).** Search Radar cue (world-frame, via radio) and Fire Control Radar position (turret-relative, when locked). Full list and noise figures in Section 5.
- **Outputs (≥ 2).** Pan/tilt motor `setPosition()` commands and a bullet launch (teleport-to-muzzle plus velocity write). Detailed in Section 3.
- **FSM with ≥ 4 states.** Five states — `IDLE`, `AIM`, `TRACK_PREDICT`, `ENGAGING`, `RESET` (Figure 1, Section 4).
- **Multi-condition decision logic.** `TRACK_PREDICT` → `ENGAGING` requires prediction error below threshold AND intercept in range AND sustained for N frames; FCR lock combines FOV-cone membership AND range. See Section 5.
- **Safety / fail-safe.** Convergence gate before firing, universal `RESET` recovery, and the anti-friendly-fire pan-angle gate. Detailed in Section 6.
- **Sensor fusion.** Heterogeneous Kalman fusion of the Search Radar ($R_{\text{SEARCH}} = 0.1$) and FCR ($R_{\text{FCR}} = 0.001$), with predict-every-tick for dropout safety. See Section 5.
- **Context-aware behaviour.** The `AttackPredictor` learns launch-sector patterns online and pre-slews the turret in `IDLE`; the convergence gate adapts to track volatility. See Section 4.
- **Real-time logic.** Synchronous Sense → Think → Act loop on the 32 ms Webots tick, with sensing and FSM evaluation running every tick. See Section 3.
- **Advanced AI / adaptive component.** Online logistic regression (`SGDClassifier`, `partial_fit` per launch) over online-discovered sectors with EMA drift tracking — one learning step and one next-sector prediction per engagement. See Section 4.

3. System Architecture

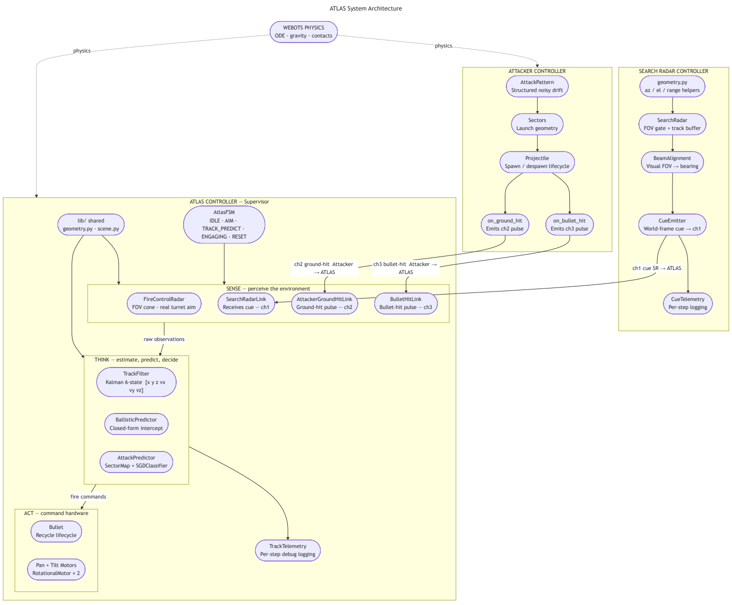

Figure 2: ATLAS system architecture diagram

Every 32 ms tick the ATLAS controller runs the three-stage Sense → Think → Act loop in order; Figure 2 groups its components by stage, with arrows showing the data that crosses each boundary. Information flows in one direction each tick: raw sensor reads → world snapshot (Sense) → filtered state + chosen action (Think) → motor and bullet writes (Act).

Sense reads the Webots devices — radio receivers, the on-board FCR, and the pan/tilt joint-angle sensors — and emits a fixed-shape world snapshot for the tick: a Search Radar cue (or none), an FCR position (when locked), any hit pulses, and the measured boresight. No interpretation happens here. Noise figures and per-stream behaviour are in Section 5.

Think takes the snapshot and produces a chosen action through three activities: **estimation** (the Kalman `TrackFilter` maintains a fused position-velocity estimate), **prediction** (the `BallisticPredictor` extrapolates an intercept and the `AttackPredictor` learns launch-sector patterns online), and **decision** (the `AtlasFSM` arbitrates via guarded transitions over those products). The output is a target pan/tilt and a fire flag. The state machine is detailed in Section 4.

Act turns that decision into hardware writes: `setPosition()` on the pan and tilt motors, and — when the FSM commits to a shot — teleport plus velocity writes on the recycled bullet `Solid`. A pan-angle friendly-fire gate sits between the FSM and the bullet to suppress shots that would cross the Search Radar's bearing.

4. Behavioural Model

ATLAS uses an FSM for autonomous target detection, tracking, prediction, and engagement. FSM control gives deterministic behaviour, modular transitions, and a guaranteed fail-safe recovery path under noisy sensing. The five states are summarised below; Figure 1 shows the full transition diagram. Sensor processing details (the Kalman filter, fusion weighting, and convergence gate) are covered in Section 5 and are referenced rather than repeated here.

`IDLE`. The turret holds its search beam, or pre-aims at the `AttackPredictor`'s next predicted sector, while waiting for a Search Radar cue. On the first cue of a new launch, the `AttackPredictor` performs one online learning step on the just-observed launch and immediately predicts the sector after that, which `IDLE` consumes for pre-aim. The FSM moves to `AIM` once a cue has been present for several consecutive frames, or to `RESET` on a ground-hit cue.

`AIM`. The turret slews toward the raw Search Radar cue; the Kalman estimate has just been reset and is not yet trustworthy, so no fusion runs. The FSM moves to `TRACK_PREDICT` as soon as the FCR locks on, or to `RESET` on a ground-hit cue.

`TRACK_PREDICT`. Each tick the turret slews to the latest filtered position, keeping the projectile inside the FCR's FOV so it returns fresh measurements — closing a tight track-and-aim loop. The filter fuses those measurements with the Search Radar cue, and the `BallisticPredictor` propagates the estimate forward to an intercept. The FSM moves to `ENGAGING` once the convergence gate trips, or to `RESET` on a ground-hit cue.

`ENGAGING`. The turret holds aim at the frozen intercept and fires a bullet, subject to the anti-friendly-fire pan-angle gate. No further prediction runs. The FSM moves to `RESET` on a bullet-hit cue, a ground-hit cue, or the target leaving range.

`RESET`. Wipes the Kalman state, clears the stale Search Radar cue, and resets the prediction history and intercept so no stale data leaks into the next engagement. The FSM then returns to `IDLE`.

5. Perception / Sensor Processing

Perception is built from radar, inter-process radio, and joint-angle sensors. Each tick the controller drains five streams: a coarse world-frame cue from the Search Radar (noise std ≈ 0.2 m), a turret-relative position from the on-board Fire-Control Radar (FCR) when locked (noise std ≈ 0.02 m), one-shot ground-hit and bullet-hit pulses, and the pan/tilt joint angles giving the measured boresight.

At the core of the pipeline is a Kalman filter — a running estimate of the projectile's position (x, y, z) and velocity $\left(v_x, v_y, v_z\right)$. The **predict** step advances the estimate each tick using projectile physics (position by velocity, gravity on velocity), so a transient dropout cannot collapse the track. The **update** step blends incoming measurements with the prediction; radars report position only, so velocity is inferred from its evolution over time. Measurement weight is governed by the noise variance R, and the two radars are separated by two orders of magnitude — $R_{\mathrm{FCR}} = 0.001$ vs $R_{\mathrm{SEARCH}} = 0.1$ — so the FCR dominates corrections. The FCR is fused on every locked tick; the Search Radar is fused only during `TRACK_PREDICT`, adding a weak long-range correction without polluting the precise track. The resulting estimate feeds the `BallisticTrajectoryPredictor`, which extrapolates to a closed-form intercept.

These products drive the FSM through compound guards. The FCR lock flag promotes `AIM` $\rightarrow$ `TRACK_PREDICT`; the fire decision is stricter, requiring the predicted-vs-observed error to stay below threshold AND the intercept to stay in range, both for several consecutive frames, before `TRACK_PREDICT` $\rightarrow$ `ENGAGING`. A final gate suppresses the shot if the measured pan angle lies inside the friendly-radar exclusion cone.

6. Safety and Robustness

ATLAS layers several safety mechanisms over the FSM and perception pipeline. The two principal ones are:

Convergence gate (fail-safe before firing). `TRACK_PREDICT` $\rightarrow$ `ENGAGING` only trips once the predicted-vs-observed error stays below threshold AND the intercept stays in range for several consecutive ticks, so a single-frame "good prediction" cannot commit the turret to a shot.

Anti-friendly-fire exclusion zone. Every fire command is gated against the measured pan angle; if the boresight lies within ± 0.4 rad ($\approx 23°$) of the Search Radar's bearing, the shot is suppressed. The turret may still track through the cone — only firing is blocked.

Supporting measures: universal `RESET` recovery (every state can exit to `RESET`, which wipes filter state before returning to `IDLE`), sensor-dropout tolerance (the Kalman predict step runs every tick regardless of measurement availability), and a debounced `IDLE` $\rightarrow$ `AIM` transition that rejects single-frame cue flickers.

7. Code Structure

The ATLAS software system is designed using modular architecture to separate all the independent components. The central layer that dictates the main process is implemented in atlas_controller.py. This is primarily responsible for constructing the system modules, wiring them together, and executing the main simulation loop.

Core perception and tracking functionalities are distributed across the FireControlRadar module (handling the narrow-field precision target tracking using field-of-view gating and range filtering) and the SearchRadarLink (receives target cues from external Search Radar controller). Projectile state estimation and prediction are handled by the TrackFilter (implements Kalman filtering) and the BallisticTrajectoryPredictor (computes future intercept locations for engagement). AtlasFSM controls the system's behaviour (managing IDLE, AIM, TRACK_PREDICT, ENGAGE, and RESET states). The adaptive behaviour is implemented through the AttackPredictor (analyses the attack-launch patterns and pre-slews the turret toward the future launch regions). Additional helper modules include: LaunchLatch, Bullet, Scoreboard, TrackTelemetry, AttackerGroundHItLink and BulletHitLink. These support projectile management, scoring, telemetry, and engagement landing.

The main control loop operates using Webot's fixed timestep execution model (robot.step(timestep) ! = −1) and during each timestep, the system follows Sense -> Process -> Decide -> Act architecture. Sense focuses on updating radar cues, projectile-hit signals, and FCR measurements. Process discusses the states of: Predict which focuses on the Kalman filter state estimate and Fuse which focuses on integrating the new radar observations into the filter. Decide advances onto the FSM and determine the tracking and engagement actions. Act is moving the turret, firing the bullet toward the intercept point and updating the engagement state.

Several external Python libraries were used throughout this project. The Webots Python API (controller.Supervisor) provides access to robot devices, simulation state, and Supervisor functionality. numpy was used for numerical calculations and geometry handling, while filterpy provides the Kalman filtering framework used in projectile tracking. scikit-learn is used within the adaptive attack-pattern learner for lightweight machine-learning classification, and standard Python libraries such as math and os are used for geometry and configuration handling. The repository also includes pytest for automated unit testing and validation.

8. Key Code Snippits

9. Results / Demonstration Summary

ATLAS successfully demonstrates autonomous projectile detection, tracking, prediction, and interception within the Webots simulation environment. The system can detect a moving projectile and track them using the Search Radar process scans which send the coarse world-frame cues to the turret. The Fire Control Radar (FCR) located on the turret only locks onto the target when turret is physically skewed in that direction. The turret can use the Kalman track filter and ballistic trajectory predictor to estimate the projectile's future path and compute the intercept point. The result is a sensor-driven engagement pipleine where real projectile motion and collisions determine whether the turret bullet (fired when all the conditions are met) hits the incoming projectile, or the projectile hits the ground.

The system successfully demonstrates multiple safety and fail-safe behaviors including confidence-gated engagement, RESET recovery, and restricted firing sectors. These mechanisms implemented in ATLAS improve the reliability and robustness of the system, especially during unexpected circumstances like unstable sensor conditions and target loss. For this project, the main challenges faced include modeling the Search Radar and FCR as 3D radars in Webots environment, figuring out the incoming projectile's motion so the prediction and intercept pipeline functioned correctly, and adding the adaptive attack-pattern feature that learns launch sectors from observed cues.

10. Individual Contributions